Simulación de SFX

Grado en Ingeniería del Software

U-tad — Curso 25/26

Laboratorio 1 (Grupal)

§ Instrucciones §

- El código del laboratorio debe organizarse en un script de Python (`.py`) por cada problema, debidamente documentado.
- Se debe incluir una presentación que explique el desarrollo del trabajo, resultados y conclusiones. Opcionalmente, se puede incluir una video-presentación.
- No está permitido el uso de código procedente de referencias o modelos de lenguaje (LLMs) como ChatGPT o Github Copilot.
- Todos los ficheros deben entregarse comprimidos en un fichero `.zip`.

Método de Jacobi para sistemas sparse

Consideremos la siguiente *matriz cíclica tridiagonal* A de tamaño $n \times n$ y el vector de unos b de tamaño n:

$$\mathbf{A} = \begin{pmatrix} 4 & -1 & 0 & \cdots & 0 & -1 \\ -1 & 4 & -1 & \cdots & 0 & 0 \\ 0 & -1 & 4 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 4 & -1 \\ -1 & 0 & 0 & \cdots & -1 & 4 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \\ 1 \end{pmatrix}$$

El objetivo es resolver el sistema $\mathbf{Ax} = \mathbf{b}$. La solución exacta es $\mathbf{x} = \left(\frac{1}{2}, \frac{1}{2}, \ldots, \frac{1}{2}\right)$ para cualquier valor de n.

Nota

Cada problema debe resolverse para valores crecientes de $n = 10, 20, 100, 200, 1000, 2000, 10000, 20000, \ldots$ hasta donde sea posible o razonable. No es necesario llegar al mismo valor de n en todos los apartados: si en algún momento el programa se queda sin memoria o tarda demasiado, basta indicarlo.

Problema 1: Enfoque denso

(1) Implementar una función `construir_A(n)` que devuelva la matriz $\mathbf{A}$ como un `np.array` de tamaño $n \times n$. *Nota: se puede utilizar `np.diag` o `np.fill_diagonal` para construir las diagonales.*

- Para cada valor de n, construir la matriz $\mathbf{A}$ y el vector $\mathbf{b}$, midiendo el tiempo de construcción con `time.perf_counter()` y la memoria ocupada por $\mathbf{A}$ con `A.nbytes`. **¿Cuánta memoria (en MB o GB) necesitaría la matriz $\mathbf{A}$ densa para $n = 100\,000$?**

(2) Utilizar la función `jacobi` disponible en `jacobi.py` para resolver el sistema, midiendo el tiempo. Calcular el error respecto a la solución exacta. *Nota: tomar $\mathbf{x}_0 = \mathbf{0}$ como vector inicial.*

(3) Resolver el sistema con `numpy.linalg.solve` para cada valor de n, midiendo el tiempo. Calcular el error respecto a la solución exacta y comparar con los tiempos del apartado anterior.

Problema 2: Jacobi para el sistema cíclico

Para este sistema concreto, las ecuaciones del método de Jacobi se pueden escribir sin hacer referencia a la matriz $\mathbf{A}$:

$$
\begin{aligned}
x_0^{(k)} &= \frac{1}{4}\left(1 + x_1^{(k-1)} + x_{n-1}^{(k-1)}\right) \\
x_i^{(k)} &= \frac{1}{4}\left(1 + x_{i-1}^{(k-1)} + x_{i+1}^{(k-1)}\right), \quad i = 1, \ldots, n-2 \\
x_{n-1}^{(k)} &= \frac{1}{4}\left(1 + x_0^{(k-1)} + x_{n-2}^{(k-1)}\right)
\end{aligned}
$$

(1) Implementar una función `jacobi_ciclico(n, tol, maxit)` que resuelva el sistema utilizando directamente estas ecuaciones, **sin construir ni almacenar la matriz A**. Los únicos parámetros de entrada deben ser n, la tolerancia y el número máximo de iteraciones.

(2) Para cada valor de n, resolver el sistema con `jacobi_ciclico`, midiendo el tiempo y calculando el error respecto a la solución exacta. Indicar además la memoria utilizada (solo se necesitan 2 vectores de tamaño n).

(3) Comparar tiempos y memoria con los resultados del Problema 1. ¿Qué conclusiones se extraen?

Problema 3: `scipy.sparse` [Opcional]

El módulo `scipy.sparse` permite almacenar matrices sparse de forma eficiente, guardando solo los elementos no nulos. El siguiente código construye la matriz $\mathbf{A}$ en formato sparse y resuelve el sistema:

```
from scipy import sparse
from scipy.sparse.linalg import spsolve

# Diagonales
diag_principal = 4.0 * np.ones(n)
diag_superior  = -1.0 * np.ones(n - 1)
diag_inferior  = -1.0 * np.ones(n - 1)

# Matriz tridiagonal
A_sparse = sparse.diags(
    [diag_inferior, diag_principal, diag_superior],
    offsets=[-1, 0, 1],
    shape=(n, n),
    format="lil"
)
# Elementos cíclicos (esquinas)
A_sparse[0, n - 1] = -1.0
A_sparse[n - 1, 0] = -1.0
A_sparse = A_sparse.tocsc()

b = np.ones(n)
x = spsolve(A_sparse, b)
```

(1) Utilizar este código para resolver el sistema para los mismos valores de n, midiendo tiempos. Comparar con los resultados de los Problemas 1 y 2.